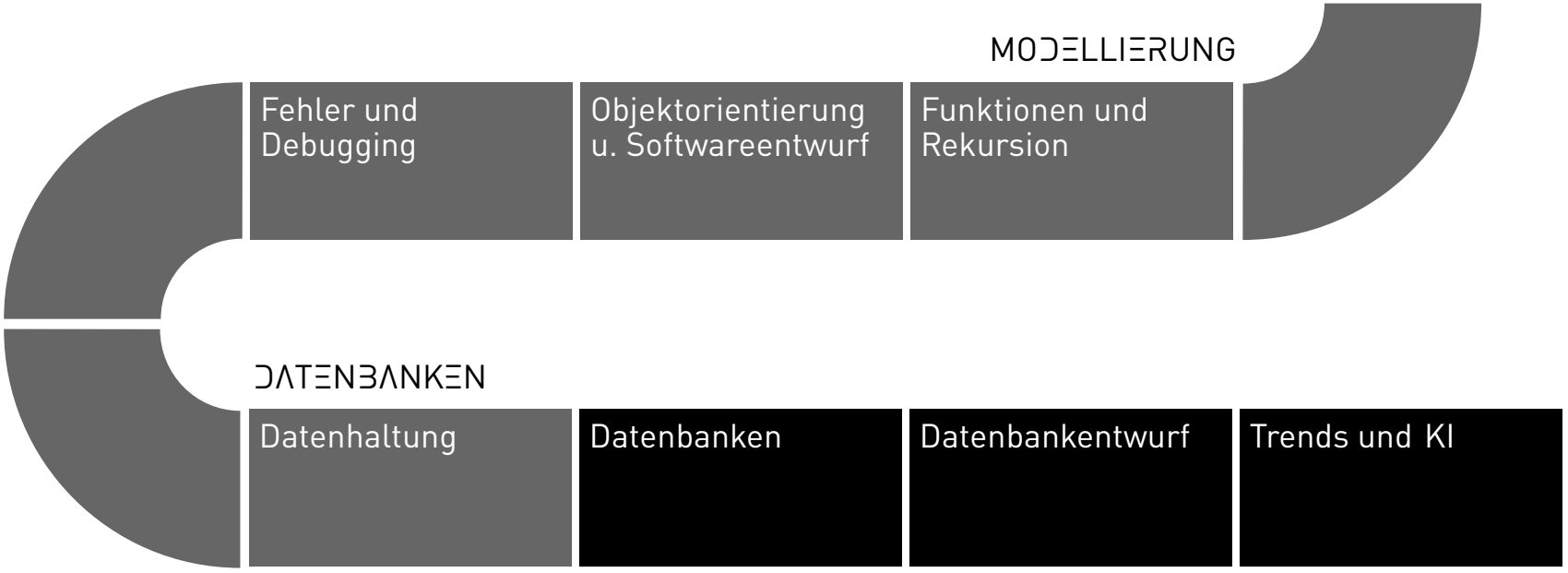
programming and databases

Joern Ploennigs

Data storage

MIDJOURNEY: LIBRARIAN, REF. GIUSEPPE ARCIMBOLDO

Process


MODELLIERUNG
Fehler und Debugging
Objektorientierung u. Softwareentwurf
Funktionen und Rekursion
DATENBANKEN
Datenhaltung
Datenbanken
Datenbankentwurf
Trends und KI

Where is our data stored?

Smartphones and Tablets

- Use: Apps that are as simple to use as possible and highly focused
- Data: Data are stored per-app on the device's built-in storage

Desktop PCs

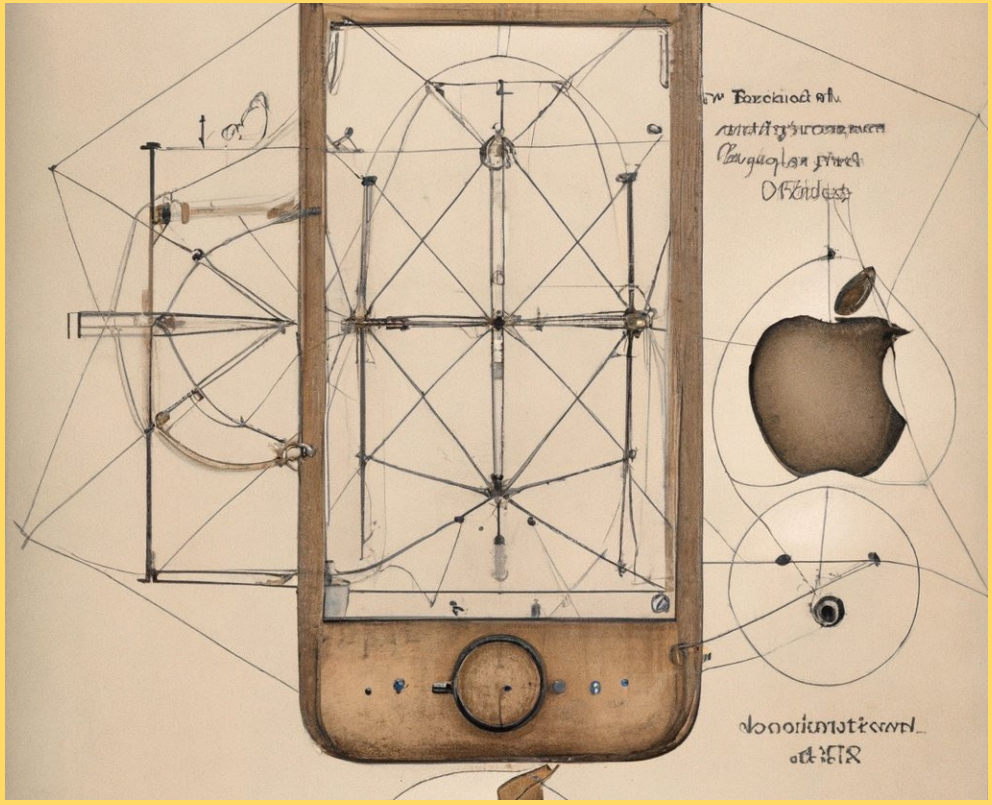
- Use: Nowadays mostly used as a workstation or hobby machine
- Data: Data reside in folder structures, stored on local drives

Web and Cloud Applications

- Use: Everywhere—from apps to high-performance computing
- Data: Data reside in globally distributed server farms, usually managed by large corporations

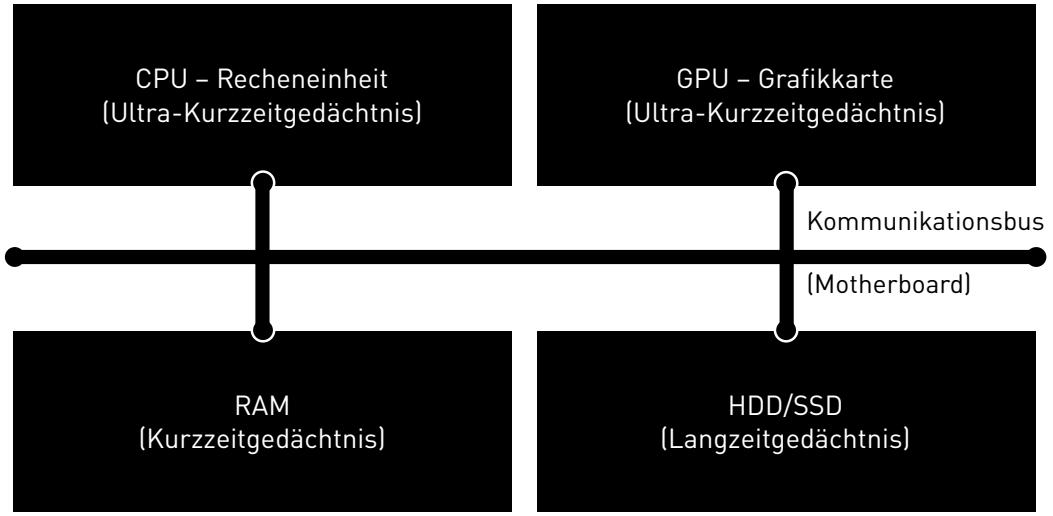
Lecture Hall Question

- What hardware makes up a computer?
- How do these compare to human memory?



DALL-E 2: Early designs of the iPhone by Leonardo da Vinci

Introduction - Where is data stored in a computer?



The computer has memory types similar to those of humans:

- CPUs and GPUs have small registers and cache memory (ultra-short-term memory)
- RAM is volatile memory, i.e., its contents are lost when the power is turned off (short-term memory)
- The HDD/SSD is non-volatile storage, i.e., the contents remain intact (long-term memory)

CPU Register & Cache - Storage at the Processor Level

- *Register* – The memory used by the CPU for calculations (very small)
- *Cache* – Here code and data are prefetched (caching), which will likely be used soon or which should be written elsewhere
- Characteristics:
  - Very fast, on-processor memory
  - Expensive, extremely small capacity
  - Must operate at roughly the same speed as the arithmetic unit to avoid creating a bottleneck

RAM - Random Access Memory

- Also known as "Direct-access memory" or "Working memory"
- Read-write memory that does not have to be read sequentially; data can be addressed directly by its address
- These accesses are fast; blocks can be addressed efficiently
- Nowadays most commonly used in the context of CPU- or GPU-near memory, i.e., it holds the data currently in use, which would be lost if power were lost

How is data organized in RAM?

- The memory cells in RAM are divided into addressable blocks
- The operating system assigns each running program as many blocks as it needs
- Programs organize these blocks into *Stack* and *Heap*:
  - *Stack* contains the function calls and important (simple) variables
  - *Heap* contains all other variables
- Each variable in code consists of:
  - a reference (pointer) to that address
  - the size of the variable in memory (determined by the data type)
  - in garbage-collected languages (Python, Java, JavaScript, etc.) there is for each variable also a counter of how often the variable is used

HDD/SSD – Hard Disk Drive / Solid State Drive

Hard Disk Drive (HDD)

- Data is stored magnetically on the storage medium
- Can last for many years
- Susceptible to shocks

Solid State Drive (SSD)

- Data is stored in electrical charges
- Can last several years, but will eventually discharge
- Erasing uses a voltage spike (flash) → Flash memory

Magnetic tapes

- Data is magnetically stored on plastic tapes
- The data can last for many decades
- Are still used for backups today
- Inexpensive, but very slow

How is data organized in long-term storage?

- The memory cells in long-term storage are also divided into addressable blocks.
- The operating system organizes these blocks and keeps track of which data is stored where (e.g., which blocks belong to which file)
- This organizational structure is called a *file system*

File System - Desktop PCs: Files in Folder Structures

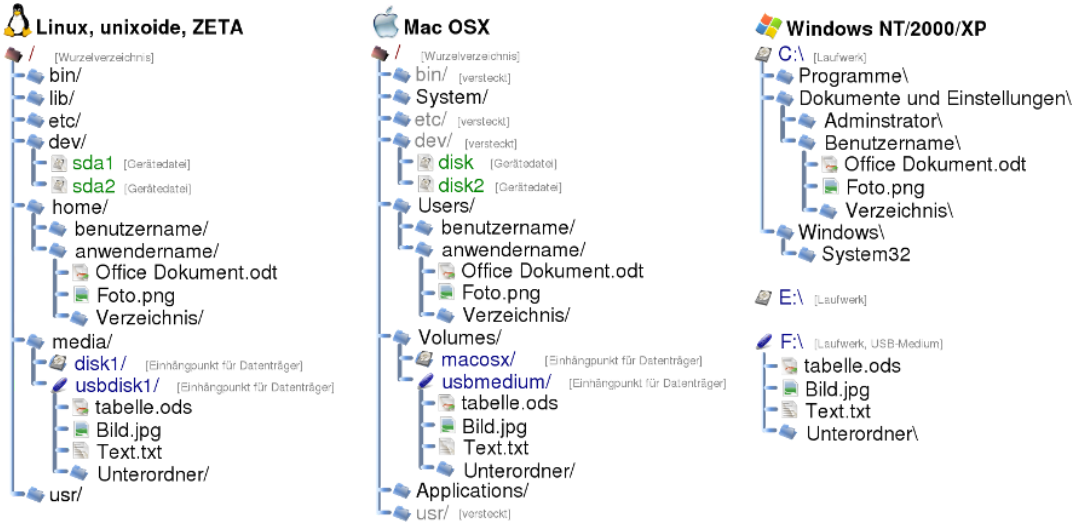
- Hierarchical (like in office filing) organized into drives (volumes), folders & files
- Data is stored in files
- These are placed in folder hierarchies (tree structures!)
- They are identified by folder paths and names

Advantages:

- This allows a very large number of files to be organized

Disadvantages:

- Can quickly become hard to navigate
- Programs can read almost anything



File System - Smartphones & Tablets: App-specific Storage

- Data are typically assigned to apps (encapsulated)
- This way the app doesn't see the full file system (and the user often doesn't either)
- This reduces the app's ability to mess things up and improves security

Advantages:

- More intuitive use with fewer files per app
- Increased security

Disadvantage:

- It is difficult to sync files between apps

File System - Cloud: Databases

- Data cannot be stored locally
- They are usually stored only in databases — see the next lecture

Advantages:

- Programs and data are physically separated
- Many program instances can access the same data
- Can be added or removed arbitrarily

Disadvantages:

- Harder to configure and debug
- A certain loss of control

File System - in Python

- Python generalizes working with file systems as much as possible
- Many different functions and libraries! `os`, `io`, `open()`, `fileinput` ...
- Different levels of abstraction – from direct string-based read operations to the hierarchical, object-oriented representations of entire directory structures
- Files are rarely read "as a whole" (inefficient for large files), but line by line, character by character, or also selectively, e.g., through a table of contents

File formats – Rough categorization

Text

- The file is a large string
- Can be read by humans and software
- Usually results in larger files
- Easier to debug since readable
- Good for structured content (e.g., text, attributes, statistics)
- Additional structure is added via syntax rules (e.g., .csv, .json, ...)

Binary

- The file is a byte array
- Only readable by software, not by humans
- Usually results in smaller files
- Harder to debug since not readable
- Good for unstructured and large content (e.g., to display images and videos)
- File extensions indicate which programs the files are associated with

Data formats for BU engineers - Type 1: Geometric Model Formats

- Vector and raster data defined using a coordinate system
- Different dimensionalities (2D, 2.5D, 3D, ...)
- Commonly used formats:
  - *2D Design Formats:* DWG, DXF, SVG, PDF, PNG, TIFF
  - *3D Design Formats:* IFC (STEP), IFC (XML), IFC (JSON), DWG, DXF, OBJ, 3DS
  - *Geospatial Data Formats:* Shapefile, GML (XML), KML (XML), GeoTIFF, GeoJSON (JSON)

Data formats for BU engineers — Type 2: Attribute formats

- Descriptive, non-geometric data for a specific context
- Often organized as tables or lists of data objects
- Commonly used formats:
  - CSV, ODF (XML), XLSX (XML), XLS, JSON
  - Different levels of complexity

Data formats for BU Engineers - Type 3: Geometry formats

- Geometric data are usually mathematical, and there is no single clearly defined path
- Graphic formats define such 'styles', e.g., for points, lines, polygons, etc.
- Commonly used formats:
  - CSS, SLD (XML), ArcGIS Styles (*.lyr)

Data Formats for Civil Engineers - Type 4: Topology Formats

- Geometry by adjacency relations instead of coordinate systems
- Nodes, edges, meshes, grids
- Commonly used formats:
  - GML (XML), TopoJSON (JSON)

Exchange formats - JSON (JavaScript Object Notation)

- A human- and machine-readable, structured data format
- Primarily used as an interchange format
- Implemented using key-value pairs (similar to Python dictionaries)
- Allows mapping of all data types from Python:
  - Number
  - String
  - Boolean
  - List
  - Dict
  - None (null)

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "children": [ ],
  "spouse": null
}
```

Exchange formats - XML (Extensible Markup Language)

- Markup languages allow marking up parts of a text to add additional (mostly machine-readable) information and semantics
- XML is in this context a meta-language that allows such languages to be defined
- Markup here is done through opening and closing tags, which are enclosed by `<` and `>`
- Examples of XML-related languages:
  - HTML
  - XHR (XML HTTP Request)
  - GML

```
<person>
  <name> John </name>
  <isAlive> true </isAlive>
  <age> 25 </age>
  <address>
    <cityStreet> New York, 21 2nd
Street </cityStreet>
    <postalCode> 10021-3100
</postalCode>
  </address>
  <children> </children>
  <spouse> </spouse>
</person>
```

Exchange formats - CSV (Comma-Separated Values)

- Textual representation of structured data (tables, lists, etc.)
- Table rows and columns are separated by delimiter characters
- Delimiter characters can be semicolon, comma, tab, etc.

```
FirstName;LastName;IsAlive;Age
John;Smith;true;25
Mary;Sue;true;30
```

Lesson Learned

Midjourney: Wilhelm Tell's son with an apple on his head, an arrow stuck in it

Lesson Learned - Goal Setting

- **S**pecific: describe the goal clearly and precisely in just two sentences.
- **M**easurable: The goal's attainment can be determined quantitatively or qualitatively.
- **A**ttractive: The attainment of the goal is desirable for YOU (I-perspective).
- **R**ealistic: The goal is ambitious and achievable.
- **T**ime-bound: A concrete deadline by which the goal should be achieved.

questions?

PROGRAMMIERUNG UND DATENBANKEN